E X P E R I M E N T R E P O R T

Fundamentals of Generative AI and Large Language Models (LLMs)

A Comprehensive Academic Report for University-Level AI/ML Coursework

Abstract

Generative Artificial Intelligence (Generative AI) has emerged as one of the most transformative paradigms in modern machine learning, enabling systems to synthesise novel and coherent content across text, images, audio, and code. This report provides a structured examination of the theoretical and practical foundations of Generative AI with a particular emphasis on Large Language Models (LLMs). The discussion is organised into four interconnected sections: (1) foundational concepts including generative versus discriminative modelling, core model families, and key mathematical intuitions; (2) architectural evolution culminating in the Transformer and its major variants — GPT, BERT, and T5; (3) real-world applications spanning natural language processing, computer vision, code generation, multimodal AI, and emerging scientific domains; and (4) the critical role of scaling — quantified by Kaplan et al. and the Chinchilla study — including emergent capabilities, alignment challenges, and efficiency techniques such as LoRA, quantisation, and Mixture of Experts.

1. Foundational Concepts of Generative AI

1.1 What Is Generative AI?

Generative AI refers to a class of machine learning systems designed to produce new data instances that are statistically consistent with a learned training distribution. Unlike conventional supervised models that map inputs to fixed labels, generative models internalise the underlying structure of data and can synthesise entirely novel samples — whether sentences, images, molecules, or musical phrases.

The fundamental objective is to approximate an unknown, high-dimensional data distribution $P_{data}(x)$ and then draw samples from this approximation. In practice, a model parameterised by θ is trained so that $P_\theta(x) \approx P_{data}(x)$, typically by maximising the likelihood of observed examples or by minimising a divergence measure between the two distributions.

1.2 Generative vs. Discriminative Models

A discriminative model learns the conditional probability P(y | x) — the probability of a label y given an input x. Classifiers such as logistic regression, support vector machines, and BERT-based fine-tuned models fall into this category. They are highly effective at classification and regression tasks but cannot generate new data.

A generative model, by contrast, learns the joint distribution P(x, y) or the marginal P(x). Once this distribution is estimated, new samples can be drawn from P(x), making generative models suitable for data augmentation, creative content production, and simulation.

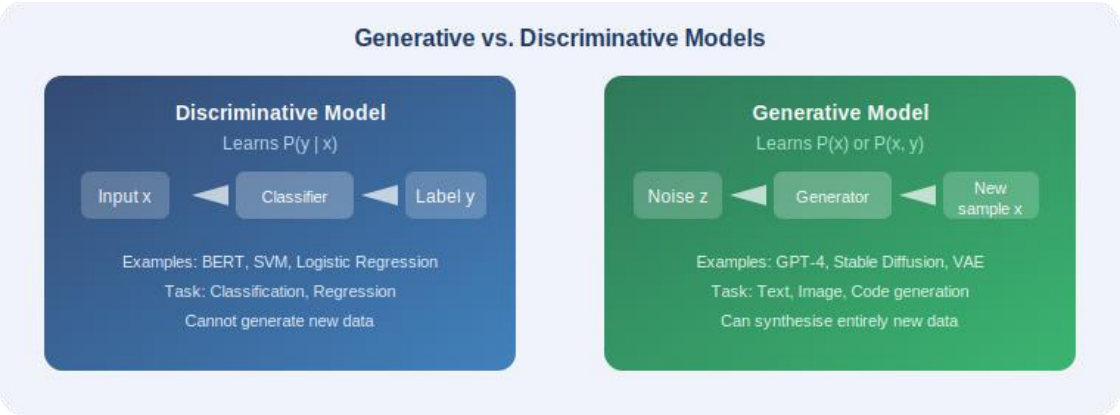

Figure 1: Generative vs. Discriminative model paradigms. Discriminative models learn P(y|x); generative models learn P(x) and synthesise new samples.

1.3 Major Families of Generative Models

Four principal architectural families have shaped the generative AI landscape. Each differs in its training objective, latent structure, and quality-diversity trade-off. The diagram below summarises these families side by side.

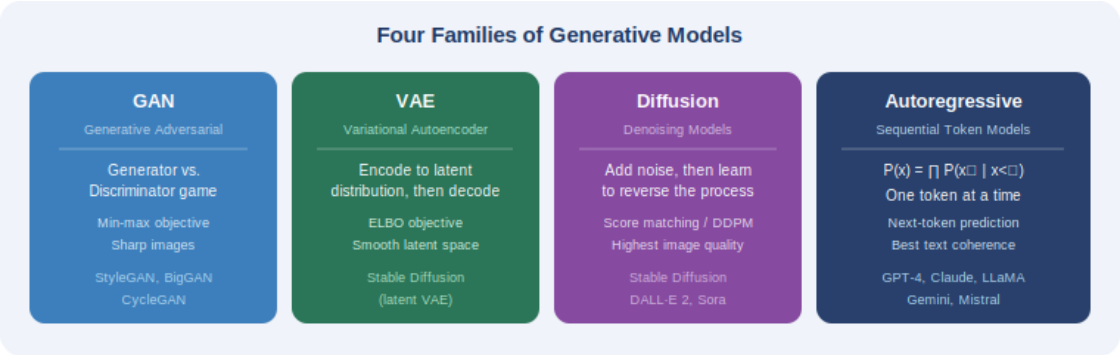

Figure 2: The four principal families of generative models — GAN, VAE, Diffusion, and Autoregressive — with their mechanisms and representative systems.

Table 1: Comparative Summary of Generative Model Families

Model Family	Core Mechanism	Training Objective	Strengths	Key Examples
GAN	Generator vs. Discriminator adversarial game	Min-max: min_G max_D V(D,G)	Sharp, high-res images; fast inference	StyleGAN, BigGAN, CycleGAN
VAE	Encode to latent distribution; decode samples	ELBO (reconstruction + KL divergence)	Smooth latent space; stable training	Kingma & Welling VAE, Stable Diffusion (latent)
Diffusion	Iterative denoising of Gaussian noise	Denoising score matching (DDPM)	Highest quality; mode coverage	Stable Diffusion, DALL·E 2, Sora
Autoregressive	Predict next token given all prior tokens	Maximum likelihood / next-token CE loss	Best text coherence; few-shot learning	GPT-4, LLaMA 3, Claude, Gemini

1.3.1 Generative Adversarial Networks (GANs)

GANs (Goodfellow et al., 2014) frame generation as a minimax game between a generator G and a discriminator D. G synthesises samples from a noise vector z ~ P(z), while D learns to distinguish real examples from generated ones. The adversarial training process drives G towards increasingly convincing outputs. Notable architectures include StyleGAN, BigGAN, and CycleGAN.

1.3.2 Variational Autoencoders (VAEs)

VAEs (Kingma & Welling, 2013) encode input data into a structured **latent space** and decode samples back into data space. The encoder outputs a mean μ and variance σ^2 defining a Gaussian distribution over z; the ELBO objective encourages accurate reconstruction and a smooth, regularised latent space.

1.3.3 Diffusion Models

Diffusion models (Ho et al., 2020) learn to reverse a gradual noise-addition process. The model is trained to denoise at each of T time steps, effectively learning to reconstruct the original distribution. At inference, a random noise vector is iteratively denoised. Stable Diffusion and DALL·E 2 achieve state-of-the-art image quality via this paradigm.

1.3.4 Autoregressive Models

Autoregressive models factorise the joint distribution as $P(x) = \prod P(x_t \mid x_1, ..., x_{t-1})$. Each token is generated conditioned on all previously generated tokens. This family — including the GPT series — produces the most coherent long-form text and dominates the LLM landscape.

1.4 Key Mathematical Intuitions

Several mathematical concepts underpin all generative modelling. Probability distributions describe the likelihood of different data configurations. The latent space is a lower-dimensional

manifold where similar data points cluster together, enabling interpolation and controlled sampling. Temperature scaling, top-k filtering, and nucleus sampling are practical techniques used to trade off diversity against quality during language model inference.

2. Generative AI Architectures — Focus on Transformers

2.1 Evolution of Sequence Models

The challenge of modelling sequential data has driven successive waves of architectural innovation. Early Recurrent Neural Networks (RNNs) maintained a hidden state across time steps but suffered from vanishing gradients. Long Short-Term Memory networks (LSTMs; Hochreiter & Schmidhuber, 1997) addressed this with gating mechanisms that selectively preserve or forget information. The attention mechanism (Bahdanau et al., 2015) allowed models to attend to any position in the input sequence, setting the stage for the fully attention-based Transformer.

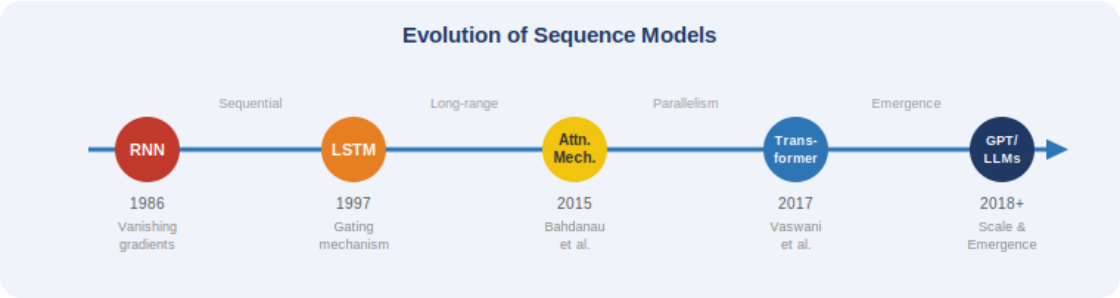

Figure 3: Evolution of sequence modelling architectures from RNNs (1986) to modern LLMs, highlighting the shift from sequential computation to parallel attention.

2.2 The Transformer Architecture

2.2.1 Self-Attention Mechanism

Self-attention allows each position in a sequence to attend to all other positions simultaneously. For a sequence of input embeddings X, three matrices are derived via learned projections: Queries (Q), Keys (K), and Values (V). The attention output is:

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) \cdot V$$

The scaling factor $\sqrt{d_k}$ prevents dot products from entering saturated softmax regions. The weighted sum of V gives the output representation, enriched by context from across the sequence.

2.2.2 Multi-Head Attention

Multi-head attention projects Q, K, and V into h different subspaces, computes attention independently in each, and concatenates the results. This allows simultaneous attention to syntactic and semantic aspects of a sequence, enabling richer contextual representations.

2.2.3 Positional Encoding

Because self-attention is permutation-invariant, explicit positional information must be injected. The original Transformer uses sinusoidal positional encodings. Modern models such as LLaMA employ Rotary Position Embeddings (RoPE), which encode relative positions and generalise better to long sequences.

2.2.4 Encoder-Decoder Structure

The original Transformer (Vaswani et al., 2017) consists of stacked encoder layers — each containing multi-head self-attention and a feed-forward sublayer with residual connections and layer normalisation — and a symmetric decoder stack that additionally includes cross-attention over the encoder output. This architecture was designed for sequence-to-sequence tasks such as machine translation.

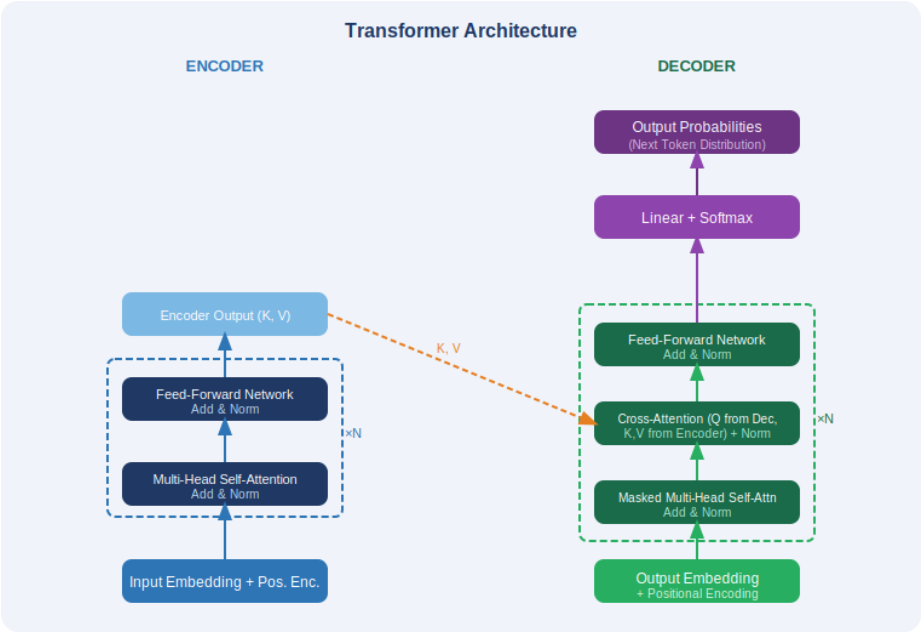

Figure 4: The Transformer architecture. The encoder (left, blue) processes the input; the decoder (right, green) generates output tokens autoregressively, attending to both its own prior outputs and the encoder's key-value representations via cross-attention (orange arrow).

2.3 Major Transformer Variants

The Transformer family has diverged into three primary architectural lineages, each optimised for a different set of tasks.

Table 2: Comparison of Major Transformer Variants

Variant	Architecture	Pre-training Task	Best For	Examples
GPT (Decoder-only)	Causal (masked) self-attention stack	Next-token prediction (left-to-right LM)	Open-ended text generation, chatbots, code	GPT-4, LLaMA, Mistral, Claude
BERT (Encoder-only)	Bidirectional self-attention stack	Masked LM + Next Sentence Prediction	Classification, NER, QA, embeddings	BERT, RoBERTa, DeBERTa
T5 (Encoder-Decoder)	Full encoder + causal decoder with cross-attn	Text-to-text span corruption objective	Translation, summarisation, seq2seq	T5, FLAN-T5, mT5

2.3.1 GPT — Decoder-Only

The GPT family uses only the decoder stack with causal (masked) self-attention. Models are pre-trained on next-token prediction, then fine-tuned or prompted. GPT-4 demonstrates sophisticated reasoning, instruction following, and multimodal understanding. The decoder-only design is the dominant architecture for open-ended text generation.

2.3.2 BERT — Encoder-Only

BERT uses only the encoder stack and is pre-trained via Masked Language Modelling. Because the encoder attends bidirectionally, BERT excels at understanding tasks — text classification, named entity recognition, question answering — but is not designed for autoregressive generation.

2.3.3 T5 — Encoder-Decoder

T5 retains the full encoder-decoder architecture and frames every NLP task as text-to-text, making the same model applicable to translation, summarisation, and QA. Instruction-tuned FLAN-T5 achieves strong zero-shot generalisation.

3. Generative AI Applications

Generative AI has achieved transformative impact across a broad range of application domains. The table below summarises major application areas, representative systems, and their societal and industrial impact.

Table 3: Generative AI Applications Across Domains

Domain	Key Tasks	Leading Models / Systems	Impact
NLP	Text generation, summarisation, translation, QA, chatbots	GPT-4, Claude, LLaMA 3, FLAN-T5, Gemini	Productivity tools, virtual assistants, search

Domain	Key Tasks	Leading Models / Systems	Impact
Computer Vision	Image synthesis, style transfer, inpainting, video gen.	Stable Diffusion, DALL·E 3, Midjourney, Sora	Creative media, design, film production
Code Generation	Completion, bug fix, test gen., transpilation	GitHub Copilot, Code Llama, Gemini Code Assist	Developer productivity, automated QA
Multimodal AI	Image captioning, visual QA, image+text generation	GPT-4V, Gemini Ultra, DALL·E 3, CLIP	Accessible interfaces, document understanding
Science & Medicine	Protein folding, drug design, materials discovery	AlphaFold 2, RFDiffusion, MusicLM, MusicGen	Drug discovery timelines, materials R&D

3.1 Natural Language Processing

Text generation is the most mature application domain. LLMs such as GPT-4, Claude, and LLaMA 3 serve as conversational assistants, drafting documents and engaging in multi-turn dialogue. Abstractive summarisation condenses articles by paraphrasing rather than extracting sentences verbatim. Neural machine translation now achieves near-human parity on high-resource language pairs. Retrieval-Augmented Generation (RAG) pipelines combine dense retrieval with generative models for factual question answering.

3.2 Computer Vision

Diffusion-based models — Stable Diffusion, Midjourney, DALL·E 3 — generate photorealistic images from natural language prompts. Style transfer algorithms recompose image content with a target artistic style using feature-level statistics. Inpainting models reconstruct missing regions of images in contextually coherent ways, with applications in photo restoration and content-aware editing. Video generation, exemplified by OpenAI's Sora, extends these capabilities to temporal sequences.

3.3 Code Generation

GitHub Copilot (OpenAI Codex), Google Gemini Code Assist, and Meta's Code Llama suggest entire function bodies and docstrings in real time. Models are evaluated on HumanEval (pass@k) and MBPP benchmarks. Beyond completion, LLMs are used for automated bug detection, unit test generation, code explanation, and cross-language transpilation.

3.4 Multimodal AI

Multimodal generative models accept and produce content across multiple modalities. GPT-4V accepts images alongside text, enabling visual question answering and document understanding.

Google's Gemini series was designed natively multimodal, processing interleaved text, images, audio, and video. These systems rely on shared embedding spaces — often trained with contrastive objectives such as CLIP — that align representations from different modalities.

3.5 Emerging Domains

In drug discovery, AlphaFold 2 predicts protein tertiary structures with atomic accuracy, while RFDiffusion proposes novel protein designs. In music generation, Google's MusicLM and Meta's MusicGen produce coherent multi-instrument compositions conditioned on text. In materials science, generative models propose crystal structures with target properties, substantially accelerating the design cycle for semiconductors and battery materials.

4. Impact of Scaling in LLMs

4.1 Scaling Laws

A landmark contribution by Kaplan et al. (2020) established that the performance of language models follows smooth power-law relationships with model size (N parameters), dataset size (D tokens), and compute budget (C FLOPs). These laws hold across many orders of magnitude, allowing practitioners to predict performance before training and to optimally allocate a fixed compute budget.

The Chinchilla study (Hoffmann et al., 2022) refined these findings by demonstrating that GPT-3-era models were significantly undertrained relative to their size. Chinchilla (70B parameters, 1.4T tokens) matched or exceeded GPT-3 (175B parameters) on most benchmarks. The key insight is that for a fixed compute budget, the optimal allocation roughly equates N and D — doubling model size should be accompanied by doubling the training data.

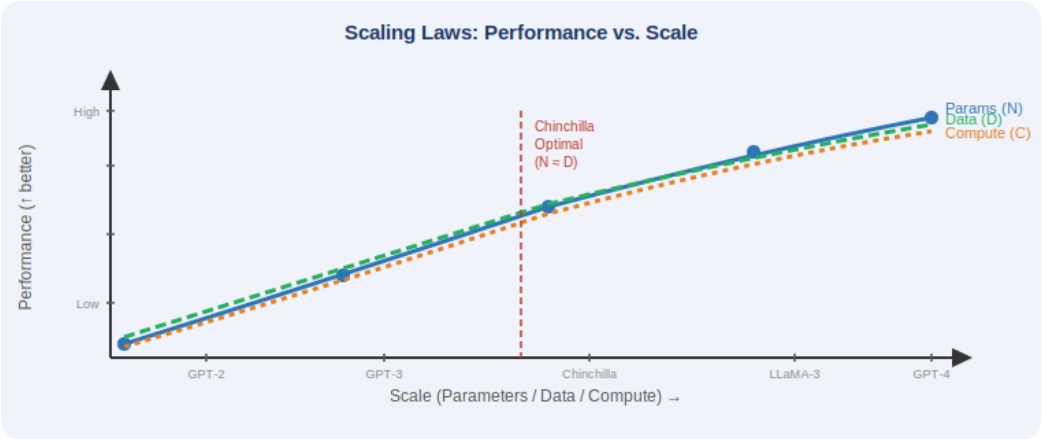

Figure 5: Scaling laws — model performance improves as a smooth power law with parameters (N), data (D), and compute (C). The Chinchilla optimal point (N ≈ D) is marked in red, corresponding to the finding that many large models are undertrained.

4.2 Emergent Abilities

One of the most striking phenomena in large-scale training is the emergence of qualitatively new capabilities that are absent at smaller scales and appear abruptly as scale crosses certain thresholds. Examples include multi-step arithmetic reasoning, chain-of-thought problem solving, few-shot learning from demonstrations provided in the prompt, and the ability to follow complex natural language instructions without explicit fine-tuning.

These emergent abilities are not fully understood theoretically; they may reflect phase transitions in the loss landscape or the accumulation of sufficient statistical evidence for rare but systematic patterns in training data. Their unpredictability complicates safety evaluation, since evaluators cannot assume that a model lacks a capability merely because smaller models did.

4.3 Challenges of Scaling

4.3.1 Compute and Environmental Cost

Training frontier LLMs requires vast computational resources. GPT-4 is estimated to have required tens of thousands of A100 GPU-months, translating to millions of dollars in cloud compute and significant carbon emissions. Inference at scale is similarly expensive. These costs concentrate frontier AI development in a small number of well-resourced organisations.

4.3.2 Hallucinations and Factual Accuracy

Despite their fluency, LLMs are prone to hallucination — generating plausible-sounding but factually incorrect statements with unwarranted confidence. This stems from the autoregressive training objective optimising surface-level coherence rather than factual correctness. Retrieval-augmented generation, factual consistency fine-tuning, and output verification systems are active mitigation strategies.

4.3.3 Alignment Challenges

Ensuring that LLMs behave in accordance with human values — the alignment problem — becomes more acute at scale. Reinforcement Learning from Human Feedback (RLHF), used in InstructGPT and Claude, fine-tunes models with human preference data to reduce harmful, dishonest, or unhelpful outputs. Direct Preference Optimisation (DPO) offers a simpler, more stable alternative.

4.4 Efficient Scaling Techniques

The following table summarises the principal efficiency techniques that allow practitioners to scale model capabilities without proportional increases in compute or memory.

Table 4: Efficient Scaling Techniques for LLMs

Technique	Core Idea	Benefit	Representative Work
Scaling Laws	Performance follows power laws in N, D, C	Predictable performance; optimal budget allocation	Kaplan et al. 2020; Chinchilla 2022

Technique	Core Idea	Benefit	Representative Work
LoRA / QLoRA	Inject low-rank matrices into frozen weights ($r \ll d$)	Fine-tune 70B models on consumer GPUs	Hu et al. 2022; Dettmers et al. 2023
Quantisation	Reduce weight precision to INT8 / INT4	3–4× faster inference; ~75% memory reduction	GPTQ, AWQ, bitsandbytes
Mixture of Experts (MoE)	Route each token to top-k of E expert FFNs	Scale parameters without scaling FLOPs	Mixtral 8×7B, Gemini, Switch Transformer
RLHF / DPO	Fine-tune on human preference data or pairwise comparisons	Safer, more helpful, better-aligned outputs	InstructGPT, Claude, Llama-2-Chat

4.4.1 Low-Rank Adaptation (LoRA)

LoRA (Hu et al., 2022) freezes the pre-trained weight matrices and injects trainable low-rank decompositions $W \approx W_0 + AB$, where $r \ll \min(d,k)$. Because only the small adapter matrices are updated, LoRA reduces trainable parameters by orders of magnitude while achieving performance competitive with full fine-tuning. QLoRA extends this by operating on quantised base models, enabling fine-tuning of 70B parameter models on consumer hardware.

4.4.2 Quantisation

Quantisation reduces the numerical precision of model weights and activations — from FP16 to INT8 or even 4-bit representations — dramatically reducing memory footprint and accelerating inference. Post-training techniques such as GPTQ and AWQ calibrate quantisation to minimise accuracy degradation. On modern hardware, quantised inference can be three to four times faster than FP16 with less than one percent performance loss on standard benchmarks.

4.4.3 Mixture of Experts (MoE)

MoE architectures replace dense feed-forward layers with E expert sub-networks and a learned routing function selecting the top-k experts for each token. Mistral's Mixtral 8×7B — eight experts of 7B parameters each, two active per token — matches or exceeds dense 70B models at a fraction of the inference cost. Google's Gemini and the reported architecture of GPT-4 are widely believed to employ MoE components.

Conclusion

This report has traced the theoretical and practical arc of Generative AI from its probabilistic foundations through the architectural revolution enabled by the Transformer and on to the societal and scientific impact of large-scale deployment. Generative models — spanning GANs, VAEs, diffusion models, and autoregressive LLMs — share a unifying ambition: to learn the statistical structure of complex data distributions and exploit that knowledge to synthesise novel content.

The Transformer architecture, with its attention-based parallelism and scalability, has proven to be the dominant substrate for this ambition, giving rise to specialised variants optimised for generation (GPT), understanding (BERT), and sequence-to-sequence conversion (T5). Real-world deployment across language, vision, code, and science has demonstrated capabilities that were implausible a decade ago, while simultaneously surfacing new challenges around factual reliability, computational cost, and value alignment.

Scaling laws provide a rigorous empirical framework for navigating the design space, but emergent capabilities and alignment risks remind us that larger is not unconditionally better. Efficiency techniques — LoRA, quantisation, and Mixture of Experts — are repositioning the frontier: capable, well-aligned models need not be prohibitively expensive to train or deploy. The trajectory of Generative AI research suggests that the coming years will be defined not merely by scale, but by the thoughtful integration of scale, efficiency, and alignment — a synthesis essential for realising the technology's transformative potential responsibly.

Key References

- Vaswani, A., et al. (2017). Attention Is All You Need. NeurIPS.
- Goodfellow, I., et al. (2014). Generative Adversarial Nets. NeurIPS.
- Kingma, D. P., & Welling, M. (2013). Auto-Encoding Variational Bayes. ICLR.
- Ho, J., et al. (2020). Denoising Diffusion Probabilistic Models. NeurIPS.
- Devlin, J., et al. (2019). BERT: Pre-training of Deep Bidirectional Transformers. NAACL.
- Kaplan, J., et al. (2020). Scaling Laws for Neural Language Models. arXiv:2001.08361.
- Hoffmann, J., et al. (2022). Training Compute-Optimal Large Language Models (Chinchilla). arXiv:2203.15556.
- Hu, E., et al. (2022). LoRA: Low-Rank Adaptation of Large Language Models. ICLR.
- Raffel, C., et al. (2020). Exploring the Limits of Transfer Learning with T5. JMLR.
- Brown, T., et al. (2020). Language Models are Few-Shot Learners (GPT-3). NeurIPS.